1. Problem Title & Link

Title: **LeetCode 435 : Non-overlapping Intervals**

Link: https://leetcode.com/problems/non-overlapping-intervals/

2. Problem Statement (Clear Explanation)

You are given an array of **intervals**, where each interval is represented as:

[start, end]

Each interval represents a **range on the number line** starting at start and ending at end.

Your task is to determine the **minimum number of intervals you must remove** so that the remaining intervals **do not overlap with each other**.

Two intervals are considered **overlapping** if:

start_i < end_j AND start_j < end_i

However, intervals that **touch at endpoints are not considered overlapping**.

Example:

[1,2] and [2,3] → not overlapping

because the first interval ends exactly where the second begins.

Your goal is to remove the **fewest possible intervals** so that the remaining intervals form a **non-overlapping set**.

3. Examples (Input → Output)

Example 1

Input

intervals = [[1,2],[2,3],[3,4],[1,3]]

Output

1

Explanation

Remove interval:

[1,3]

Remaining intervals:

[1,2], [2,3], [3,4]

No overlaps remain.

Example 2

Input

intervals = [[1,2],[1,2],[1,2]]

Output

2

Explanation

Only one interval can remain.

Example 3

Input

intervals = [[1,2],[2,3]]

Output

0

Explanation

Intervals do not overlap.

4. Constraints

1 ≤ intervals.length ≤ 100000

intervals[i].length = 2

-50000 ≤ start_i < end_i ≤ 50000

Additional notes:

- Intervals can appear **in any order**
- Negative values are allowed
- Efficient solution required for **large input sizes**

5. Core Concept (Pattern / Topic)

Greedy

Interval Scheduling

6. Thought Process (Step-by-Step Explanation)

Step 1 — Understand the problem

We want to **remove minimum intervals** so that the remaining intervals **do not overlap**.

Equivalent idea:

Instead of minimizing removals, we can:

maximize number of non-overlapping intervals

Then:

removals = total_intervals - kept_intervals

Step 2 — Greedy Insight

To maximize non-overlapping intervals:

Sort intervals by **ending time**.

Why?

Choosing intervals with **earlier end times** leaves more space for future intervals.

Step 3 — Greedy Strategy

1. Sort intervals by end
2. Pick the first interval
3. Track the end of the last chosen interval
4. If the next interval starts **after or equal to end**, keep it
5. Otherwise → remove it

7. Visual / Intuition Diagram

Example

Intervals

[1,2] [2,3] [3,4] [1,3]

Sorted by end:

[1,2] [2,3] [1,3] [3,4]

Selection:

[1,2] ✔

[2,3] ✔

[1,3] ✘ remove

[3,4] ✔

Removals = **1**

8. Pseudocode (Language Independent)

```
function eraseOverlapIntervals(intervals):

  sort intervals by end

  prev_end = intervals[0][1]

  remove_count = 0

  for each interval from index 1:

    if interval.start < prev_end:
      remove_count += 1
    else:
      prev_end = interval.end
```

```
    return remove_count
```

9. Code Implementation

Python

```
def eraseOverlapIntervals(intervals):

    intervals.sort(key=lambda x: x[1])

    prev_end = intervals[0][1]

    remove_count = 0

    for start, end in intervals[1:]:

        if start < prev_end:
            remove_count += 1
        else:
            prev_end = end

    return remove_count
```

Java

```
import java.util.*;

class Solution {

    public int eraseOverlapIntervals(int[][] intervals) {

        Arrays.sort(intervals, (a, b) -> a[1] - b[1]);

        int prevEnd = intervals[0][1];
        int remove = 0;

        for(int i = 1; i < intervals.length; i++) {

            if(intervals[i][0] < prevEnd) {
```

```
            remove++;
        }
        else {
            prevEnd = intervals[i][1];
        }
    }

    return remove;
  }
}
```

10. Time & Space Complexity

Time Complexity

O(n log n)

Sorting dominates runtime.

Space Complexity

O(1)

Only a few variables used.

11. Common Mistakes / Edge Cases

Common mistakes:

1. Sorting by **start time instead of end time**
2. Using incorrect overlap condition

Correct condition:

start < prev_end

3. Forgetting intervals that **touch at endpoints**

Example:

[1,2] and [2,3] → valid

12. Detailed Dry Run

Example:

intervals = [[1,2],[2,3],[3,4],[1,3]]

Sorted by end:

[1,2], [2,3], [1,3], [3,4]

Interval	Condition	Action	Removed
[1,2]	start	keep	0

[2,3]	$2 \geq 2$	keep	0
[1,3]	$1 < 3$	remove	1
[3,4]	$3 \geq 3$	keep	1

Result:

1

13. Common Use Cases (Real-Life / Interview)

Interval scheduling appears in:

- **Meeting room allocation**
- **CPU scheduling**
- **Calendar conflict resolution**
- **Booking systems**

14. Common Traps (Important!)

1. Sorting by **start instead of end**
2. Updating prev_end incorrectly
3. Confusing **remove vs keep logic**

15. Builds To (Related LeetCode Problems)

Progression:

LC 252 Meeting Rooms

LC 435 Non-overlapping Intervals

LC 452 Minimum Arrows to Burst Balloons

LC 56 Merge Intervals

LC 57 Insert Interval

16. Alternate Approaches + Comparison

Approach	Time	Notes
Greedy (sort by end)	O(n log n)	Optimal
DP	$O(n^2)$	Too slow

17. Why This Solution Works (Short Intuition)

Choosing intervals with the **earliest finishing time** leaves maximum room for future intervals, ensuring the maximum number of intervals can be kept.

18. Variations / Follow-Up Questions

Possible interview extensions:

1. Return **the intervals kept**, not just count
2. Find **maximum number of non-overlapping intervals**
3. Handle **weighted intervals**
4. Solve **interval scheduling with priorities**.